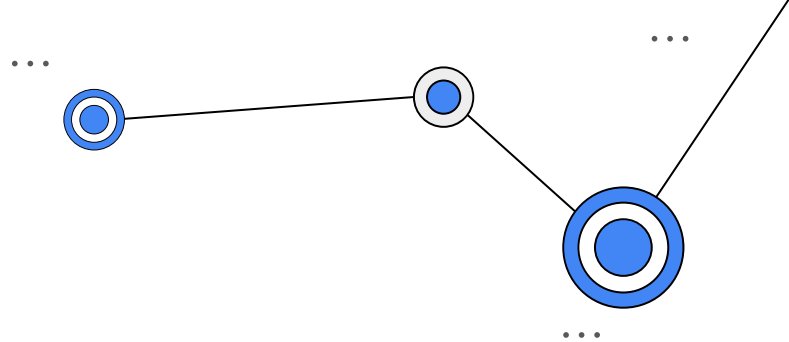




UNIVERSITÀ
DEGLI STUDI
DI TRIESTE

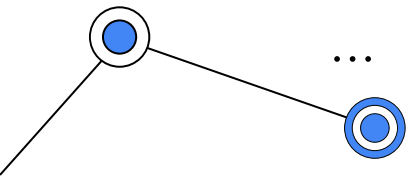


440MI

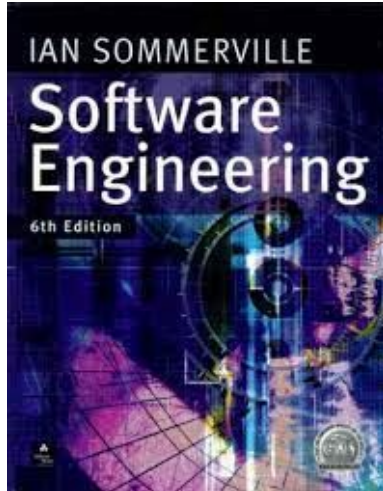
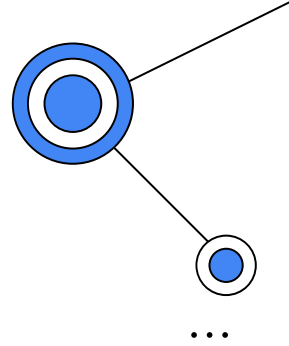
DATA-DRIVEN SYSTEMS ENGINEERING

L12 - Agile Software Development

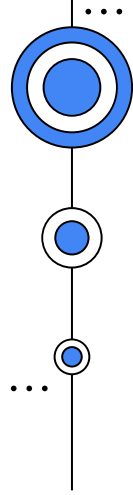
Sylvio Barbon Junior
sylvio.barbonjunior@units.it



Last lesson



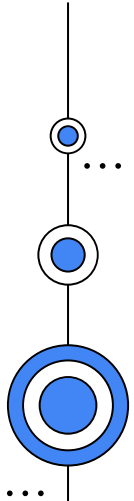
... Software Processes



12

Software Engineering

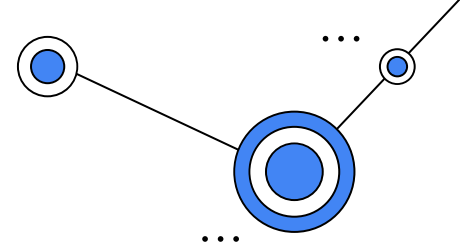
Agile Software Development



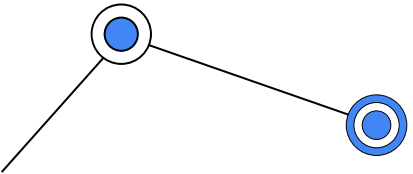
Today's goal:

- Part 1
 - Agile methods
 - Plan-driven and agile development
- Part 2
 - Extreme programming
 - Agile project management
 - Scaling agile methods

SE 12 - Agile Software Development



Part 1



SE 12 - Agile Software Development

Rapid software development

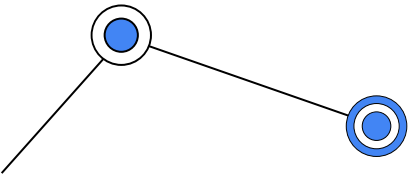
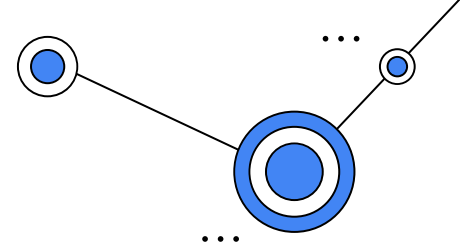
✧ **Rapid development** and **delivery** is now often the most important requirement for software systems:

Businesses operate in a **fast –changing requirement** and it is practically **impossible** to produce a set of **stable** software **requirements**

Software has to evolve quickly to reflect changing business **needs**.

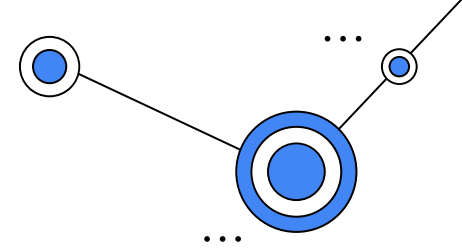
✧ **Rapid software development**

- 1) **Specification, design and implementation** are interleaved;
- 2) **System is developed as a series of versions** with stakeholders involved in version evaluation
- 3) **User interfaces** are often developed using an **IDE and graphical toolset**.

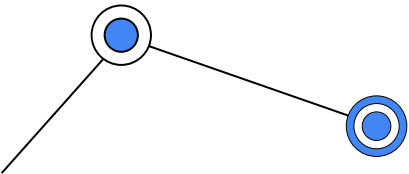


SE 12 - Agile Software Development

Agile methods

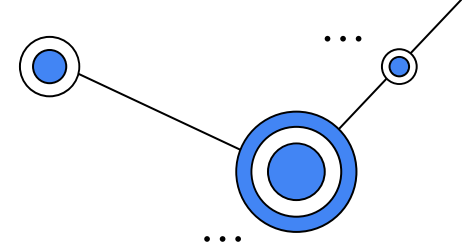


- ✧ **Dissatisfaction** with the **overheads** involved in **software design methods** of the 1980s and 1990s led to the creation of **agile methods**. These methods:
 - 1) Focus on the **code** rather than the **design**;
 - 2) Are based on an **iterative approach** to software development;
 - 3) Are intended to **deliver** working software **quickly** and evolve this quickly to meet **changing requirements**;
- ✧ **The aim of agile methods is to reduce overheads in the software process** (e.g. by limiting documentation) and to be able to respond quickly to changing requirements without excessive rework.



SE 12 - Agile Software Development

Agile manifesto



✧ We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

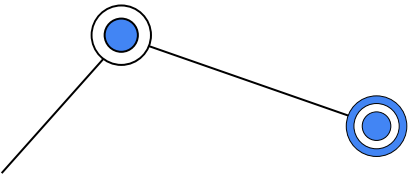
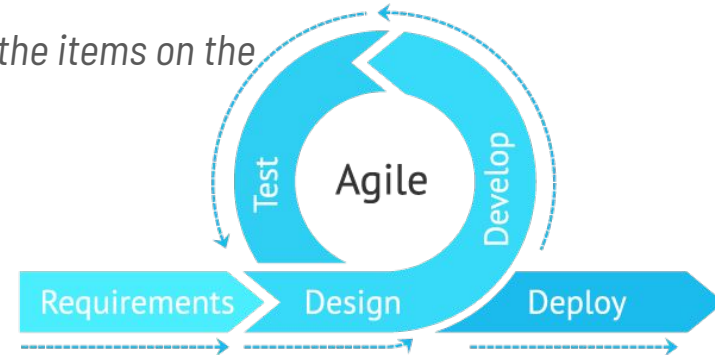
Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

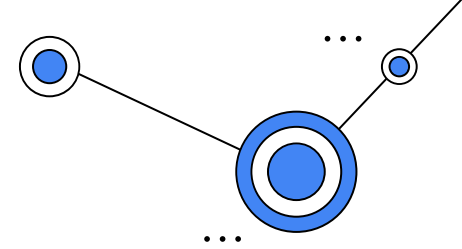
Responding to change over following a plan

✧ That is, while there is value in the items on the right, we value the items on the left more.



SE 12 - Agile Software Development

The principles of agile methods



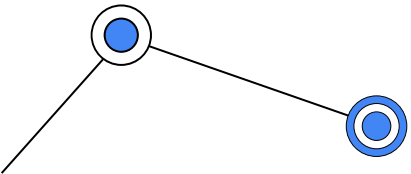
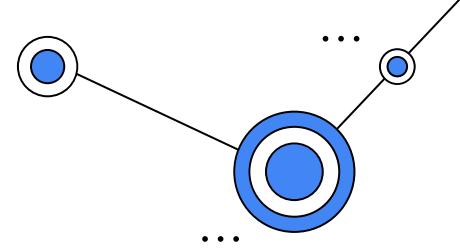
Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.
Incremental delivery	The software is developed in increments with the customer specifying the requirements to be included in each increment .
People not process	The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.
Embrace change	Expect the system requirements to change and so design the system to accommodate these changes.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process . Wherever possible, actively work to eliminate complexity from the system.



SE 12 - Agile Software Development

Agile method applicability

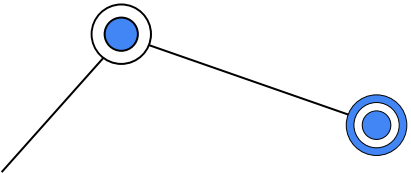
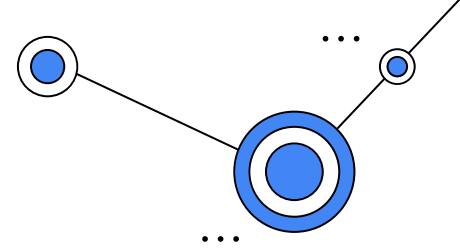
- ✦ Product development where a software company is developing a **small** or **medium-sized product for sale**.
- ✦ **Custom system development within an organization**, where there is a clear **commitment** from the **customer** to become **involved** in the development process and where there are not a lot of external rules and regulations that affect the software.
- ✦ Because of their focus on small, tightly-integrated teams, **there are problems in scaling agile methods to large systems**.



SE 12 - Agile Software Development

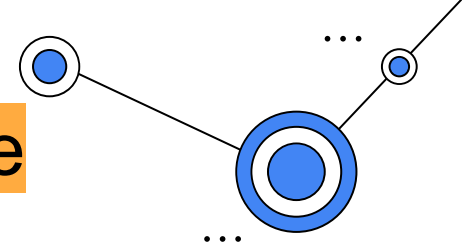
Problems with agile methods

- ✧ It can be difficult to **keep the interest of customers** who are involved in the process.
- ✧ **Team members may be unsuited** to the intense involvement that characterises **agile methods**.
- ✧ **Prioritising changes can be difficult** where there are multiple stakeholders.
- ✧ **Maintaining simplicity requires extra work.**
- ✧ **Contracts may be a problem** as with other approaches to iterative development.



SE 12 - Agile Software Development

Agile methods and software maintenance

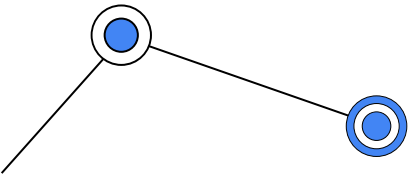


✧ **Most organizations spend** more on **maintaining existing** software **than** they do on **new software** development. So, if agile methods are to be successful, they have to support maintenance as well as original development.

✧ Two **key issues**:

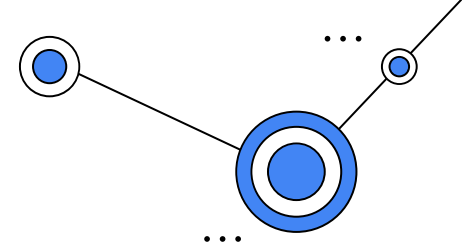
- * Are systems that are developed using an agile approach maintainable, given the emphasis in the development process of **minimizing formal documentation**?
- * **Can agile methods be used effectively** for **evolving** a system **in response to customer** change requests?

✧ **Problems may arise if original development team cannot be maintained.**



SE 12 - Agile Software Development

Plan-driven and agile development



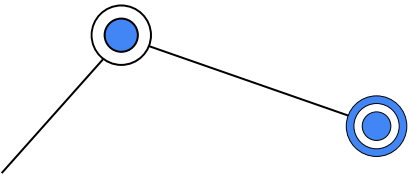
✧ Plan-driven development

A plan-driven approach to software engineering is based around **separate development stages with the outputs to be produced** at each of these stages **planned in advance**.

Not necessarily waterfall model – plan-driven, incremental development is possible
Iteration occurs within **activities**.

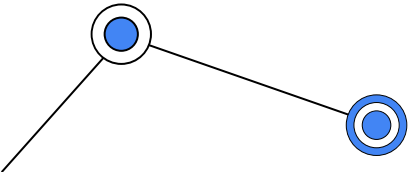
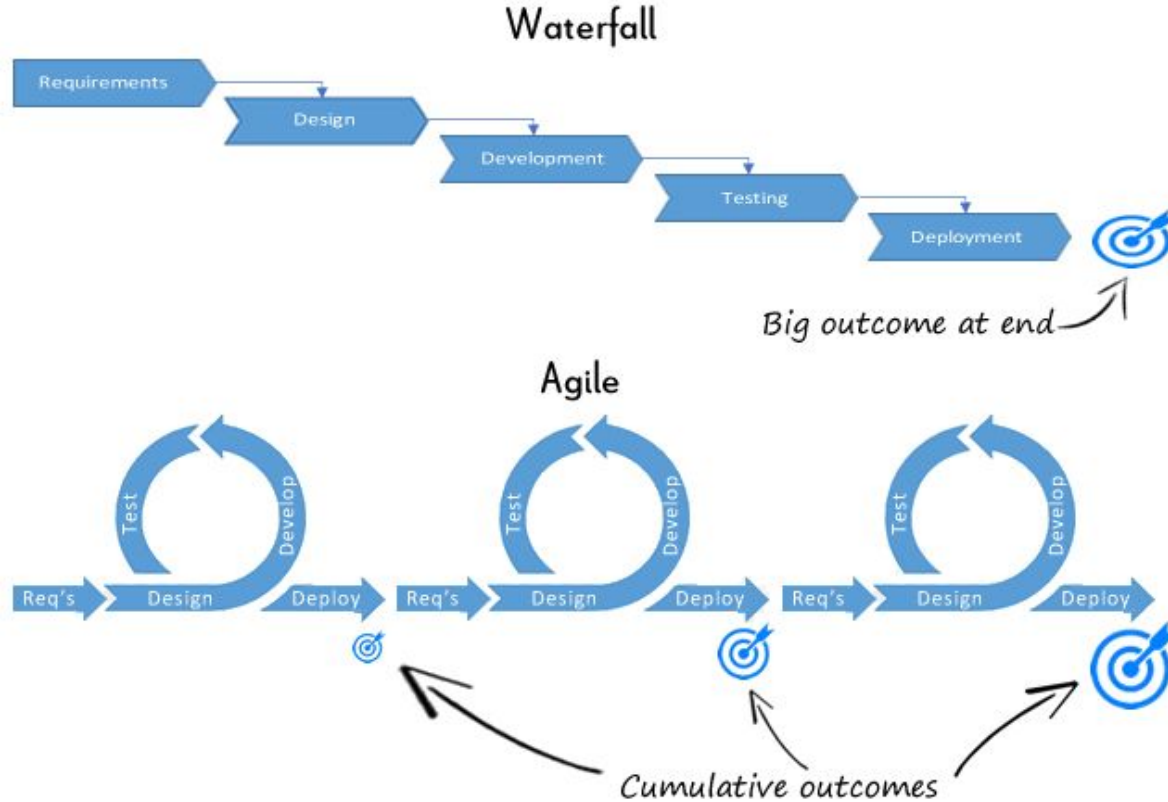
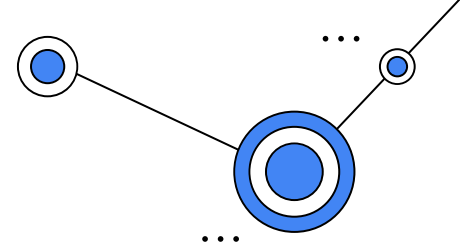
✧ Agile development

Specification, design, implementation and testing are interleaved and the outputs from the development process are decided through a process of negotiation during the software development process.



SE 12 - Agile Software Development

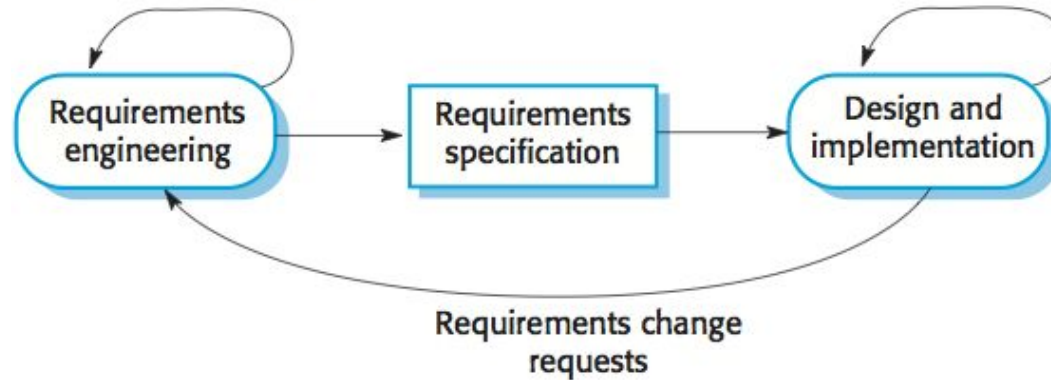
Plan-driven and agile development



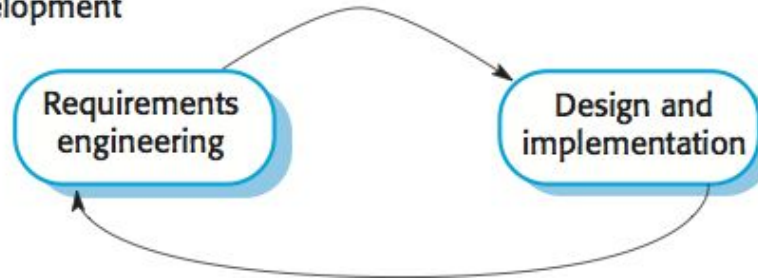
SE 12 - Agile Software Development

Plan-driven and agile specification

Plan-based development

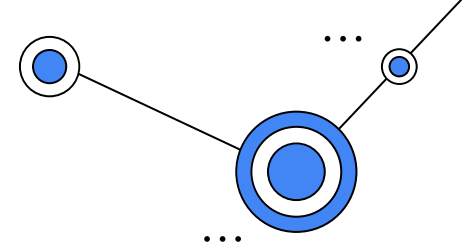


Agile development



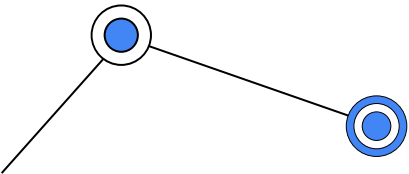
SE 12 - Agile Software Development

Technical, human, organizational issues



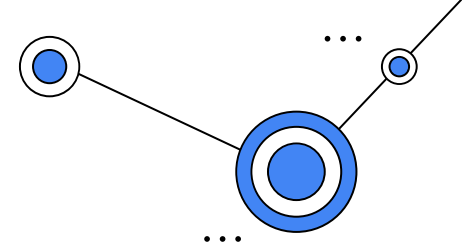
✧ **Most projects include elements of plan-driven and agile processes. Deciding** on the balance depends on:

- 1) Is it important to have a **very detailed specification** and **design** before moving to implementation? If so, you probably need to use a **plan-driven approach**.
- 2) Is an incremental delivery strategy, where you **deliver the software to customers** and **get rapid feedback** from them, realistic? If so, consider using **agile methods**.
- 3) **How large is the system that is being developed?** **Agile** methods are most effective when the system can be developed with a **small co-located team** who can communicate **informally**. This may not be possible for large systems that require larger development teams so a plan-driven approach may have to be used.



SE 12 - Agile Software Development

Technical, human, organizational issues



4) What type of system is being developed?

Plan-driven approaches may be required for systems that **require a lot of analysis** before implementation (e.g. **real-time system with complex timing requirements**).

5) What is the expected system lifetime?

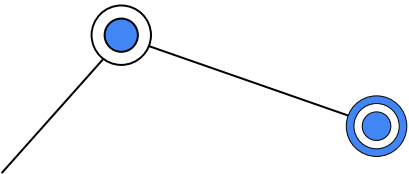
Long-lifetime systems may require more design documentation to communicate the original intentions of the system developers to the support team.

6) What technologies are available to support system development?

Agile methods rely on **good tools** to **keep track** of an evolving design

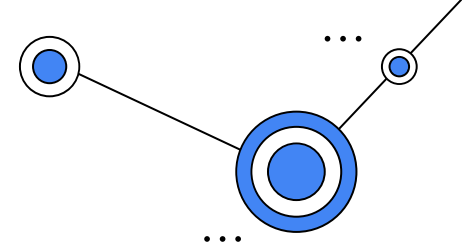
7) How is the development **team organized**?

If the development **team is distributed** or if part of the development is being outsourced, then you may need to develop **design documents to communicate** across the development teams.



SE 12 - Agile Software Development

Technical, human, organizational issues



8) Are there **cultural** or **organizational issues** that may **affect** the system development?

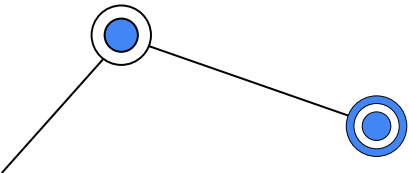
Traditional engineering organizations have a culture of **plan-based development**, as this is the norm in engineering.

9) How good are the **designers** and **programmers** in the development team?

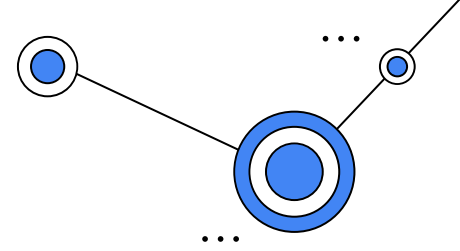
It is sometimes argued that **agile methods require higher skill levels** than plan-based approaches in which programmers simply translate a detailed design into code

10) Is the system subject to external regulation?

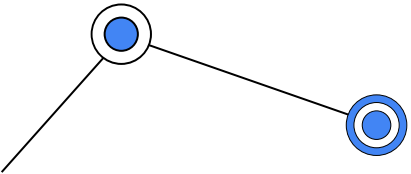
If a system has to be approved by an external regulator (e.g. the FAA approve software that is critical to the operation of an aircraft) then you will probably be required to **produce detailed documentation** as part of the system safety case.



SE 12 - Agile Software Development



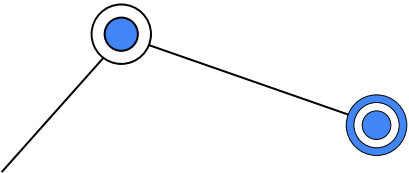
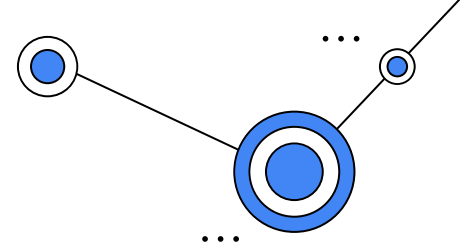
Part 2



SE 12 - Agile Software Development

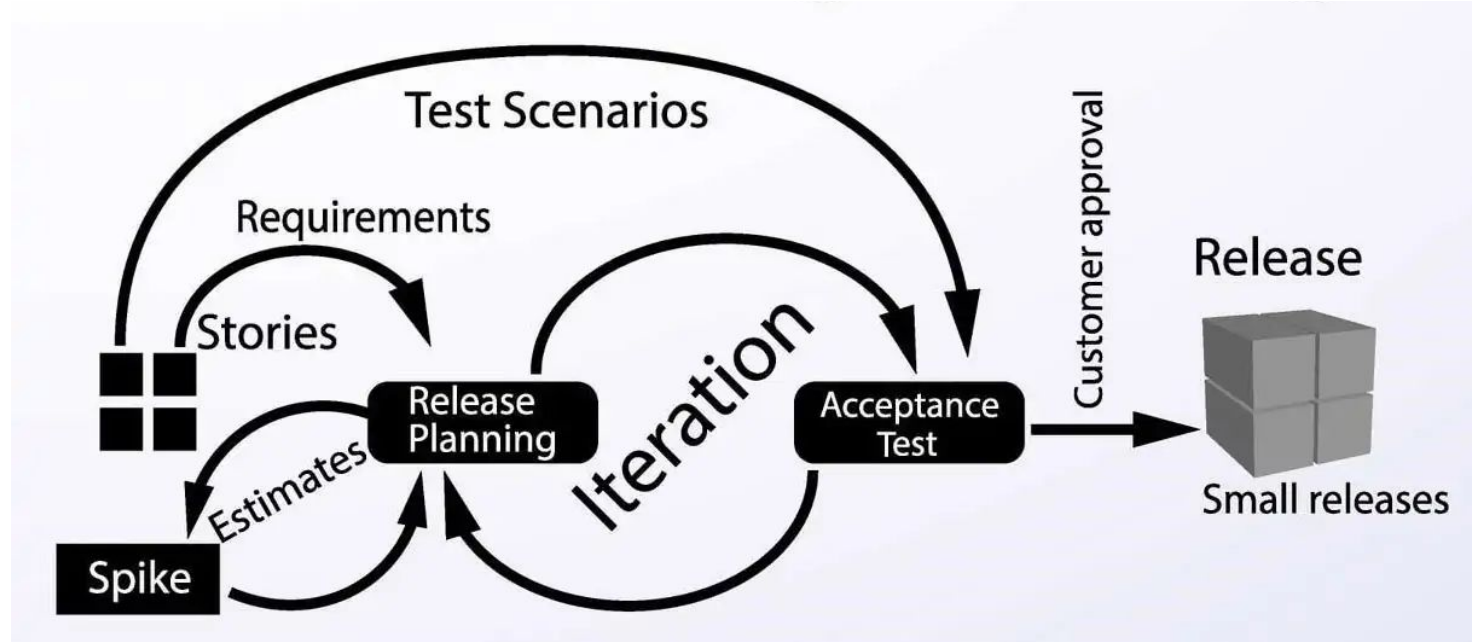
Extreme programming

- ✧ Perhaps the **best-known** and most widely used agile method.
- ✧ Extreme Programming (XP) takes an 'extreme' approach to iterative development.
 - New versions** may be built **several times per day**;
 - Increments** are delivered to customers **every 2 weeks**;
 - All tests must be run** for every build and the build is only accepted if tests run successfully.



SE 12 - Agile Software Development

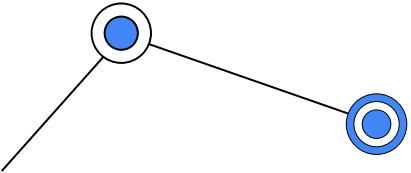
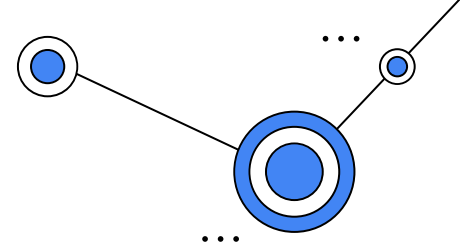
Extreme programming



SE 12 - Agile Software Development

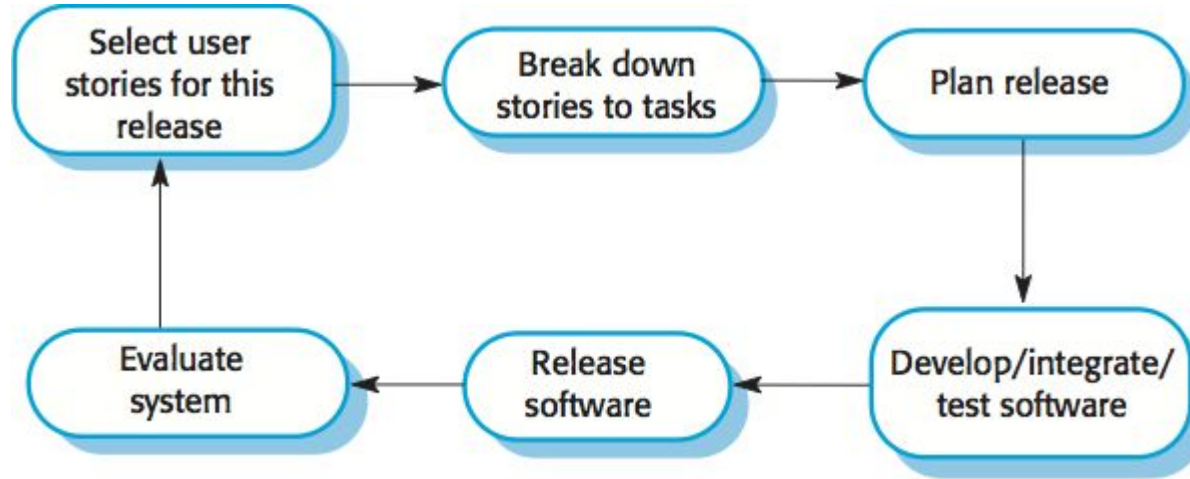
XP and agile principles

- ✧ **Incremental development** is supported through **small, frequent** system **releases**.
- ✧ Customer involvement means **full-time customer engagement** with the team.
- ✧ **People not process** through **pair programming, collective ownership** and a **process that avoids long working hours**.
- ✧ **Change supported** through **regular system releases**.
- ✧ **Maintaining simplicity** through constant refactoring of code.



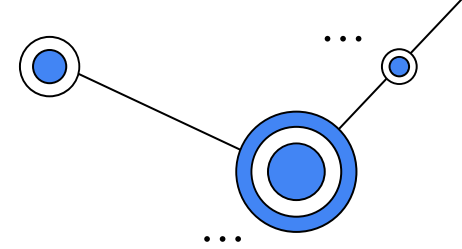
SE 12 - Agile Software Development

The extreme programming release cycle



SE 12 - Agile Software Development

Extreme programming practices (a)

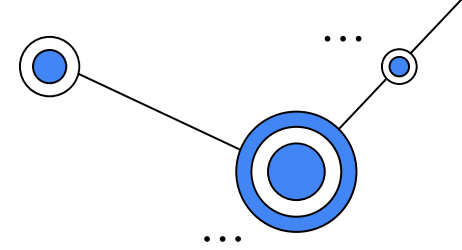


Principle or practice	Description
Incremental planning	Requirements are recorded on story cards and the stories to be included in a release are determined by the time available and their relative priority . The developers break these stories into development 'Tasks'.
Small releases	The minimal useful set of functionality that provides business value is developed first . Releases of the system are frequent and incrementally add functionality to the first release.
Simple design	Enough design is carried out to meet the current requirements and no more.
Test-first development	An automated unit test framework is used to write tests for a new piece of functionality before that functionality itself is implemented.
Refactoring	All developers are expected to refactor the code continuously as soon as possible code improvements are found. This keeps the code simple and maintainable.



SE 12 - Agile Software Development

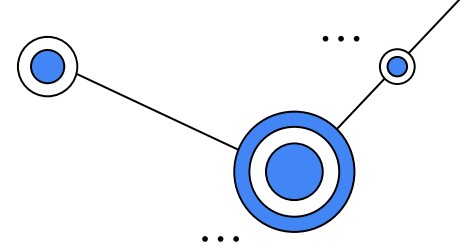
Extreme programming practices (b)



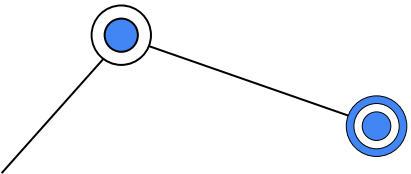
Pair programming	Developers work in pairs, checking each other's work and providing the support to always do a good job.
Collective ownership	The pairs of developers work on all areas of the system , so that no islands of expertise develop and all the developers take responsibility for all of the code . Anyone can change anything.
Continuous integration	As soon as the work on a task is complete, it is integrated into the whole system. After any such integration, all the unit tests in the system must pass.
Sustainable pace	Large amounts of overtime are not considered acceptable as the net effect is often to reduce code quality and medium term productivity
On-site customer	A representative of the end-user of the system (the customer) should be available full time for the use of the XP team. In an extreme programming process, the customer is a member of the development team and is responsible for bringing system requirements to the team for implementation.

SE 12 - Agile Software Development

Requirements scenarios

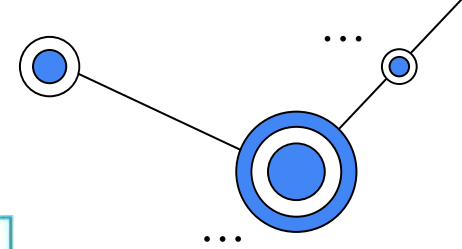


- ✧ In XP, a customer or user is part of the XP team and is responsible for making decisions on requirements.
- ✧ **User requirements** are expressed as scenarios or **user stories**.
- ✧ **These are written on cards and the development team break them down into implementation tasks.** These tasks are the basis of schedule and cost estimates.
- ✧ **The customer chooses the stories for inclusion in the next release** based on their priorities and the schedule estimates.



SE 12 - Agile Software Development

A 'prescribing medication' story



Prescribing medication

The record of the patient must be open for input. Click on the medication field and select either 'current medication', 'new medication' or 'formulary'.

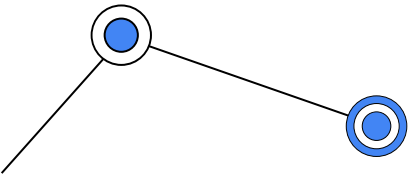
If you select 'current medication', you will be asked to check the dose; If you wish to change the dose, enter the new dose then confirm the prescription.

If you choose, 'new medication', the system assumes that you know which medication you wish to prescribe. Type the first few letters of the drug name. You will then see a list of possible drugs starting with these letters. Choose the required medication. You will then be asked to check that the medication you have selected is correct. Enter the dose then confirm the prescription.

If you choose 'formulary', you will be presented with a search box for the approved formulary. Search for the drug required then select it. You will then be asked to check that the medication you have selected is correct. Enter the dose then confirm the prescription.

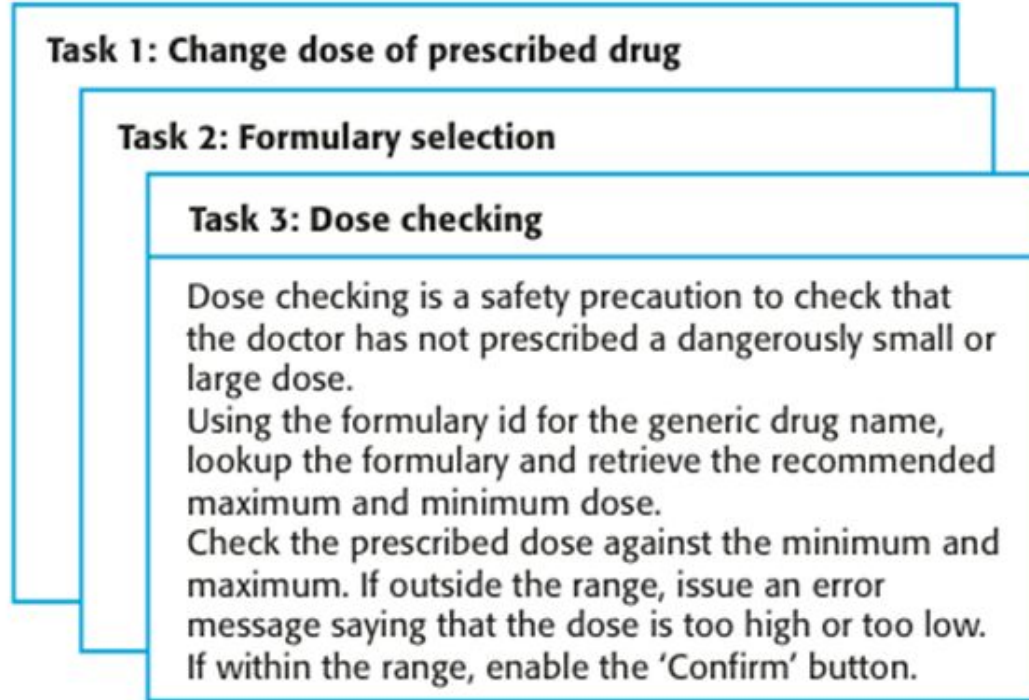
In all cases, the system will check that the dose is within the approved range and will ask you to change it if it is outside the range of recommended doses.

After you have confirmed the prescription, it will be displayed for checking. Either click 'OK' or 'Change'. If you click 'OK', your prescription will be recorded on the audit database. If you click 'Change', you reenter the 'Prescribing medication' process.



SE 12 - Agile Software Development

Examples of task cards for prescribing medication



SE 12 - Agile Software Development

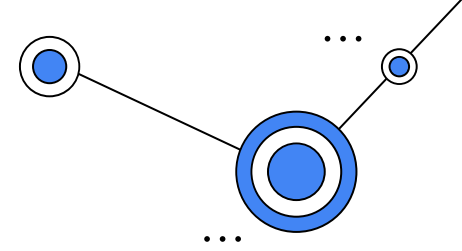
Examples of task cards for prescribing medication

Title	Size
Value statement	
Basic requirements	
Acceptance criteria (can also go on the back)	

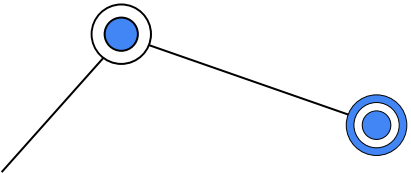
READY	IN PROGRESS	DONE
As a Mother, I want you teenagers to pick up wet towels from the floor, so that I don't go off at you.	As a Mother, I want you teenagers to stack breakfast bowls in the dishwasher, so that the cat doesn't jump onto the bench to lick them clean.	As the cat in the house, I want you to remember to get to the supermarket, so that I don't have to have leftover frozen human meals for my dinner.

SE 12 - Agile Software Development

XP and change



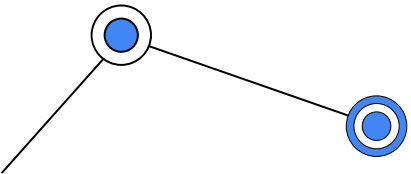
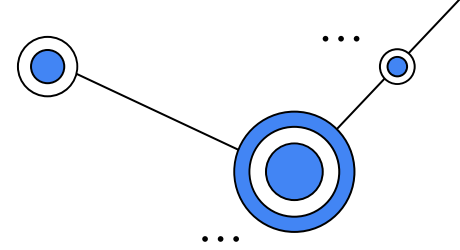
- ✧ **Conventional wisdom in software engineering is to design for change.** It is worth spending time and effort anticipating changes as this reduces costs later in the life cycle.
- ✧ XP, however, **maintains that this is not worthwhile** as changes cannot be reliably anticipated.
- ✧ Rather, **it proposes constant code improvement (refactoring)** to make changes easier when they have to be implemented.



SE 12 - Agile Software Development

Refactoring

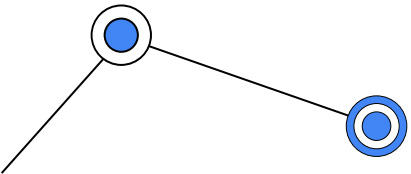
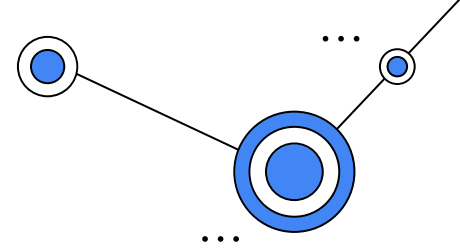
- ✧ **Programming team look for possible software improvements** and make these improvements even where there is no immediate need for them.
- ✧ **This improves the understandability of the software and so reduces the need for documentation.**
- ✧ **Changes are easier** to make because the code is **well-structured and clear.**
- ✧ However, some **changes requires architecture** refactoring and this is much more **expensive.**



SE 12 - Agile Software Development

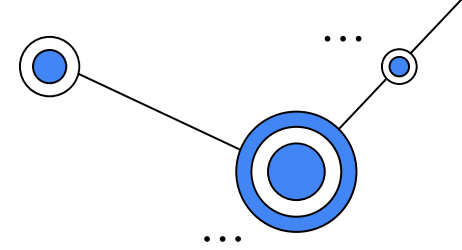
Testing in XP

- ✧ **Testing is central to XP** and XP has developed an approach where the **program is tested after every change has been made.**
- ✧ **XP testing features:**
 - 1) Test-first development.
 - 2) Incremental test development from scenarios.
 - 3) User involvement in test development and validation.
 - 4) Automated test harnesses are used to run all component tests each time that a new release is built.

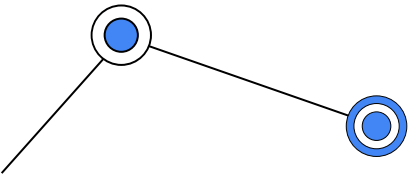


SE 12 - Agile Software Development

Test-first development



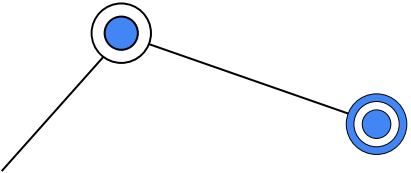
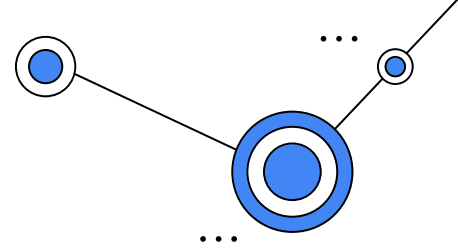
- ✦ **Writing tests before code clarifies** the requirements to be implemented.
- ✦ **Tests are written as programs** rather than data so that they can be executed automatically. The test includes a check that it has executed correctly.
Usually relies on a testing framework such as **Junit**.
- ✦ **All previous and new tests are run automatically** when new functionality is added, thus checking that the new functionality has not introduced errors.



SE 12 - Agile Software Development

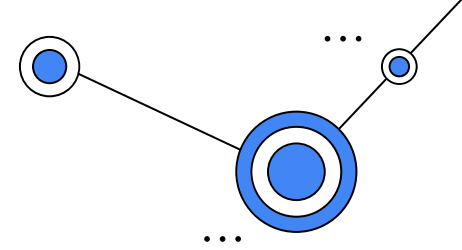
Customer involvement

- ✦ **The role of the customer in the testing process** is to help develop acceptance tests for the stories that are to be implemented in the next release of the system.
- ✦ The **customer** who is part of the **team writes tests as development proceeds**. All new code is therefore validated to ensure that it is what the customer needs.
- ✦ **However, people adopting the customer role have limited time available** and so cannot work full-time with the development team. They may feel that providing the requirements was enough of a contribution and so may be reluctant to get involved in the testing process.



SE 12 - Agile Software Development

Test case description for dose checking



Test 4: Dose checking

Input:

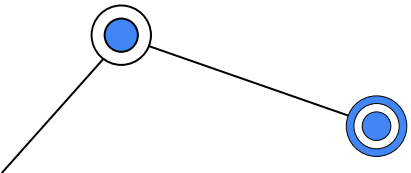
1. A number in mg representing a single dose of the drug.
2. A number representing the number of single doses per day.

Tests:

1. Test for inputs where the single dose is correct but the frequency is too high.
2. Test for inputs where the single dose is too high and too low.
3. Test for inputs where the single dose * frequency is too high and too low.
4. Test for inputs where single dose * frequency is in the permitted range.

Output:

OK or error message indicating that the dose is outside the safe range.

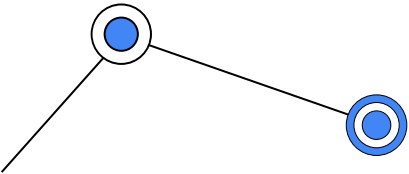
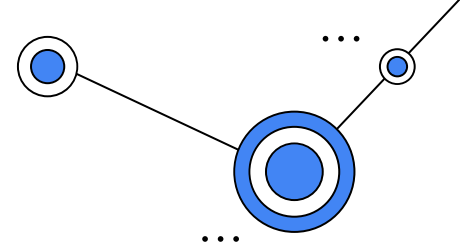


SE 12 - Agile Software Development

Test automation

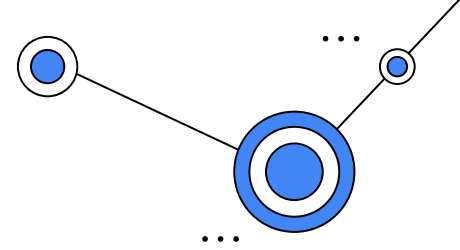
- ✧ **Test automation means** that **tests** are written **as executable components** before the task is implemented
- ✧ **As testing is automated**, there is always a set of tests that can be **quickly** and **easily executed**

Whenever any functionality is added to the system, the tests can be run and problems that the new code has introduced can be caught immediately.

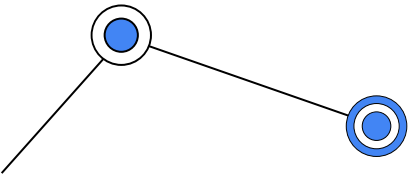


SE 12 - Agile Software Development

XP testing difficulties



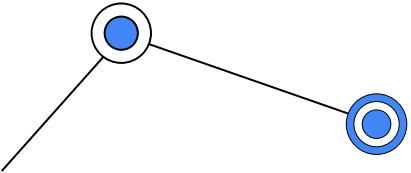
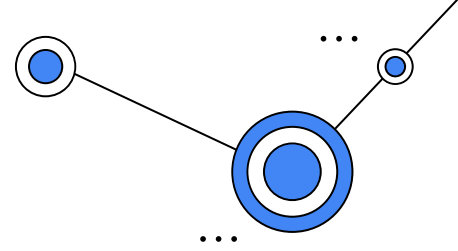
- ✦ **Programmers prefer programming to testing and sometimes** they take shortcuts when writing tests. For example, they **may write incomplete tests** that do not check for all possible exceptions that may occur.
- ✦ **Some tests can be very difficult to write incrementally.** For example, in a complex user interface, it is often difficult to write unit tests for the code that implements the 'display logic' and workflow between screens.
- ✦ **It difficult to judge the completeness of a set of tests.** Although you may have a lot of system tests, your test set may not provide complete coverage.



SE 12 - Agile Software Development

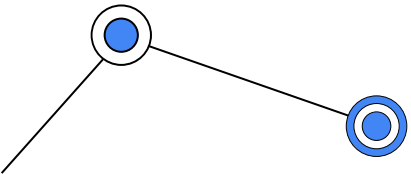
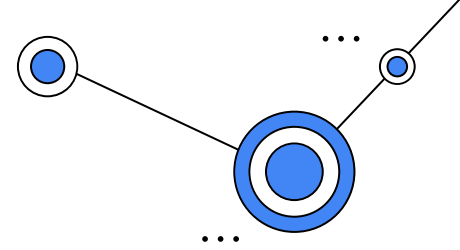
Pair programming

- ✧ **In XP, programmers work in pairs, sitting together to develop code.**
- ✧ This helps develop common ownership of code and spreads knowledge across the team.
- ✧ **It serves as an informal review process** as each line of code is looked at by more than 1 person.
- ✧ **It encourages refactoring** as the whole team can benefit from this.



SE 12 - Agile Software Development

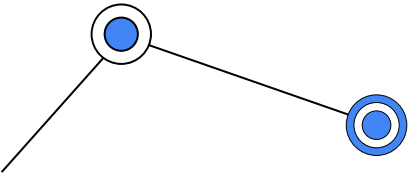
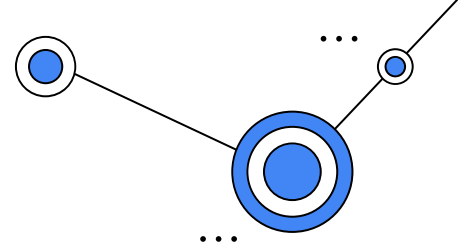
Pair programming



SE 12 - Agile Software Development

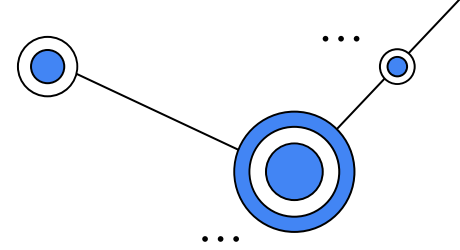
Pair programming

- ✧ In pair programming, programmers sit together **at the same workstation** to develop the software.
- ✧ **Pairs are created dynamically** so that all team members work with each other during the development process.
- ✧ **The sharing of knowledge** that happens during pair programming **is very important** as it reduces the overall risks to a project when team members leave.
- ✧ **Pair programming is not necessarily inefficient and there is evidence that a pair working together is more efficient than 2 programmers working separately.**



SE 12 - Agile Software Development

Advantages of pair programming



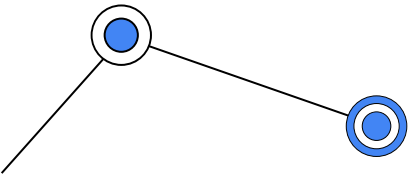
- ✧ **It supports the idea of collective ownership** and responsibility for the system.

Individuals are not held responsible for problems with the code. Instead, the team has collective responsibility for resolving these problems.

- ✧ **It acts as an informal review process because each line of code is looked at by at least two people.**

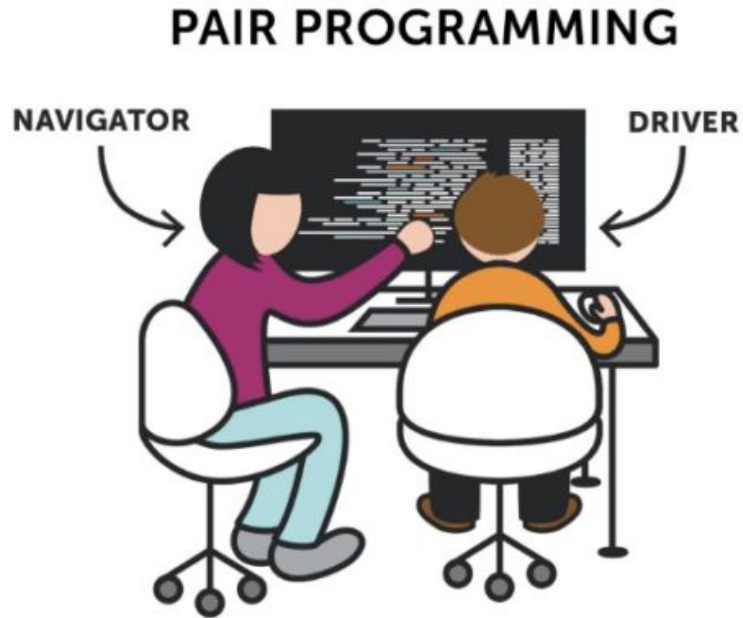
- ✧ **It helps support refactoring**, which is a process of software improvement.

Where pair programming and collective ownership are used, others benefit immediately from the refactoring so they are likely to support the process.



SE 12 - Agile Software Development

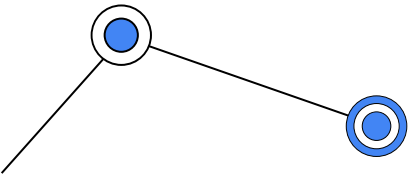
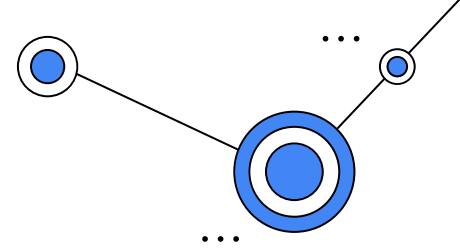
Advantages of pair programming



SE 12 - Agile Software Development

Agile project management

- ✧ The principal responsibility of software project managers is to manage the project so that the software is **delivered on time and within the planned budget** for the project.
- ✧ **The standard approach to project management is plan-driven.** Managers draw up a plan for the project showing what should be delivered, when it should be delivered and who will work on the development of the project deliverables.
- ✧ Agile project management requires a different approach, which is adapted to incremental development and the particular strengths of agile methods.



SE 12 - Agile Software Development

The end!

